

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250117>

Outline : ML tools, setting up the development environment

- ML Tools
 - a. Scikit-Learn
 - b. Keras
 - c. TensorFlow
 - d. Jupyter
- Setting up ML development environment with Jupyter
- First examples - getting and looking at the data (first steps)
 - a. Getting data
 - b. Discover and visualize data

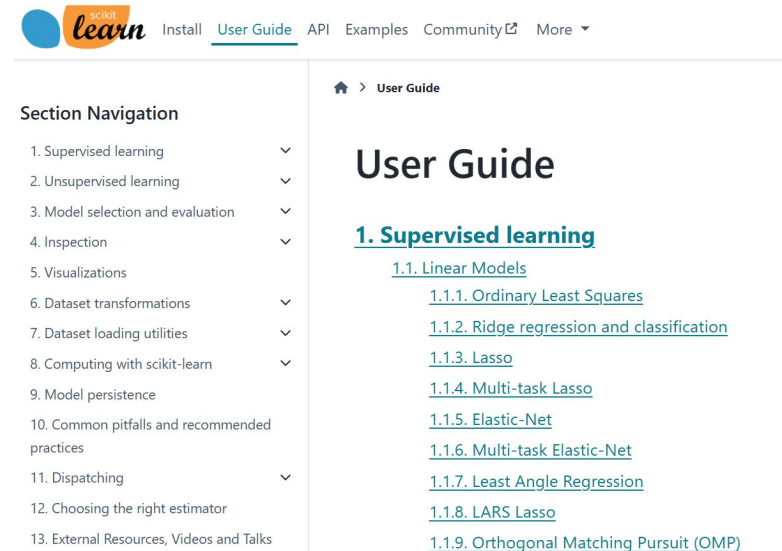
—

- **Source:** Aurelien Geron, “Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems”, 2019 ([Available online](#)) / **Chapter 2**

ML Tools

Scikit-Learn

- “a community project, developed by a large group of people, all across the world.”
- “Python module integrating a wide range of state-of-the-art machine learning algorithms for medium-scale supervised and unsupervised problems.”
- <https://scikit-learn.org/stable/index.html>



The screenshot shows the Scikit-Learn User Guide website. At the top, there is a navigation bar with links for 'Install', 'User Guide' (which is highlighted), 'API', 'Examples', 'Community', and 'More'. Below the navigation bar, the page title 'User Guide' is displayed. On the left side, there is a 'Section Navigation' sidebar with a list of 13 items, each with a dropdown arrow. The main content area on the right shows the '1. Supervised learning' section, with a list of sub-sections: '1.1. Linear Models', '1.1.1. Ordinary Least Squares', '1.1.2. Ridge regression and classification', '1.1.3. Lasso', '1.1.4. Multi-task Lasso', '1.1.5. Elastic-Net', '1.1.6. Multi-task Elastic-Net', '1.1.7. Least Angle Regression', '1.1.8. LARS Lasso', and '1.1.9. Orthogonal Matching Pursuit (OMP)'.

Scikit-Learn

Install User Guide API Examples Community More

Home > User Guide

User Guide

1. Supervised learning

- 1.1. Linear Models
 - 1.1.1. Ordinary Least Squares
 - 1.1.2. Ridge regression and classification
 - 1.1.3. Lasso
 - 1.1.4. Multi-task Lasso
 - 1.1.5. Elastic-Net
 - 1.1.6. Multi-task Elastic-Net
 - 1.1.7. Least Angle Regression
 - 1.1.8. LARS Lasso
 - 1.1.9. Orthogonal Matching Pursuit (OMP)

Keras

- “deep learning framework works with [TensorFlow](#), [JAX](#), and [PyTorch](#) interchangeably”
- https://keras.io/getting_started/



About Keras

Getting started

Introduction to Keras for engineers

Keras 3 benchmarks

The Keras ecosystem

Frequently Asked Questions

Developer guides

Code examples

Keras 3 API documentation

Keras 2 API documentation

KerasTuner: Hyperparam Tuning

KerasHub: Pretrained Models

Search Keras documentation...



► Getting started with Keras

Getting started with Keras

Learning resources

Are you a machine learning engineer looking for a Keras introduction one-pager? Read our guide [Introduction to Keras for engineers](#).

Want to learn more about Keras 3 and its capabilities? See the [Keras 3 launch announcement](#).

Are you looking for detailed guides covering in-depth usage of different parts of the Keras API? Read our [Keras developer guides](#).

Are you looking for tutorials showing Keras in action across a wide range of use cases? See the [Keras code examples](#): over 150 well-explained notebooks demonstrating Keras best practices in computer vision, natural language processing, and generative AI.

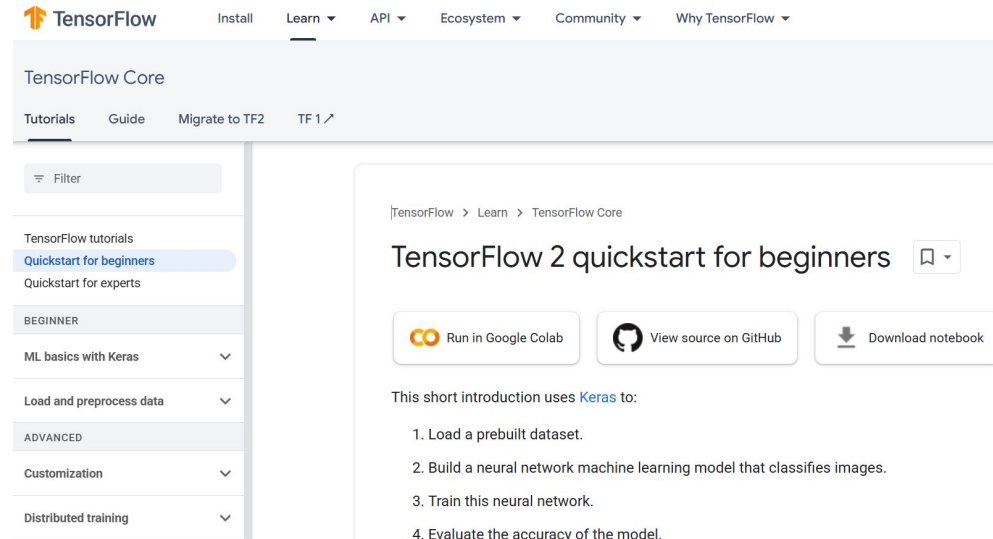
Installing Keras 3

You can install Keras from PyPI via:

```
pip install --upgrade keras
```

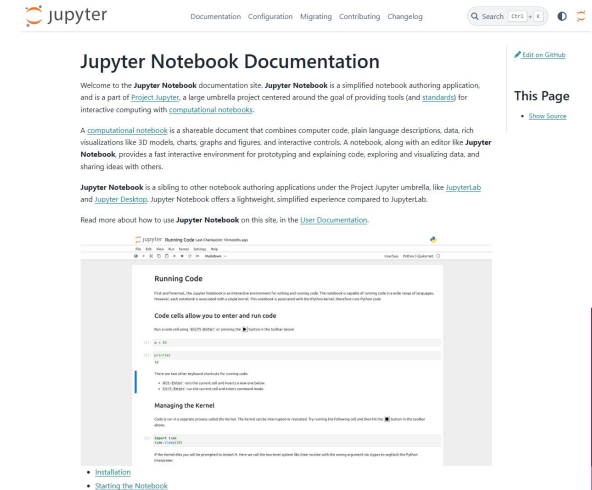
TensorFlow

- “a software library for machine learning and artificial intelligence. It can be used across a range of tasks, but is used mainly for training and inference of neural networks.”
- <https://www.tensorflow.org/learn>



Jupyter

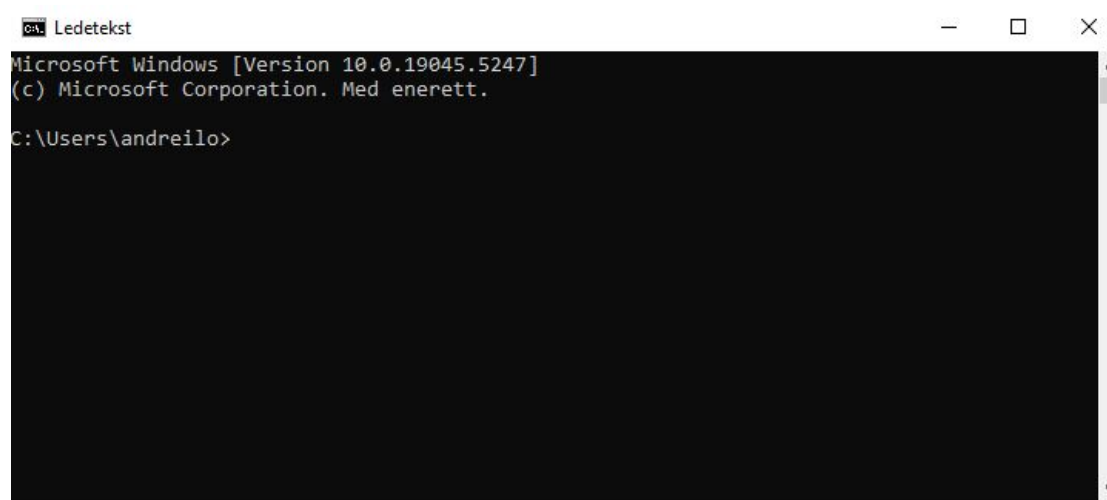
- “web-based interactive development environment for notebooks, code, and data. Its flexible interface allows users to configure and arrange workflows in data science, scientific computing, computational journalism, and machine learning.”
- <https://jupyter.org/>
- <https://docs.jupyter.org/en/latest/>
- <https://jupyter-notebook.readthedocs.io/en/latest/>



Setting up ML work environment

Command Prompt

Start a command prompt



Create a workspace (1)

Windows (start Command Prompt by “cmd” command):

```
set ML_PATH="<YOUR PATH HERE>\ml"  
mkdir %ML_PATH%
```

```
set PATH=%PATH%;<Path to CURR installation of Python>\Python311\Scripts\
```

(The version of the python as well as folder name, e.g. ‘Python311’, can be different in your case as these are dependent on YOUR installation)

Creating isolated environment (*Needs to be used once if setting up on YOUR machine*)

```
python -m pip install --user -U virtualenv  
pushd %ML_PATH%  
python -m virtualenv my_env
```

Create a workspace (2)

Windows (start Command Prompt by “cmd” command):

To activate this environment from the %ML_PATH% folder:

.\my_env\Scripts\activate

To deactivate, type:

deactivate

Installing tools

(Needs to be used once when setting up on YOUR machine)

```
python -m pip install -U jupyter matplotlib numpy pandas scipy scikit-learn
```

Registering venv to Jupyter and giving it a name:

```
python -m ipykernel install --user --name=python
```

Run Jupyter and do example in Chapter 2

jupyter notebook



BOOK

Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems

Geron, Aurelien

2019

“ Through a series of recent breakthroughs, deep learning has boosted the entire field of machine learning...”

[Available online](#)

TOP

Jupyter notebooks as deliverables

*.ipynb files / interactive python notebook or a text-based files used by Jupyter Notebook (+ eventually, data files, if these were used/stored locally) shall be delivered along with your assignment reports.

Also, JupyterHub at NTNU

“a multi-user server that allows users to access and run Jupyter Notebooks through a web browser.”

<https://www.ntnu.no/wiki/display/jupyterhub/>

Trying book examples, some steps

2019 edition: <https://github.com/ageron/handson-ml2/tree/master>

2022 edition: <https://github.com/ageron/handson-ml3/tree/main>

Framing the problem

Build a model of housing prices

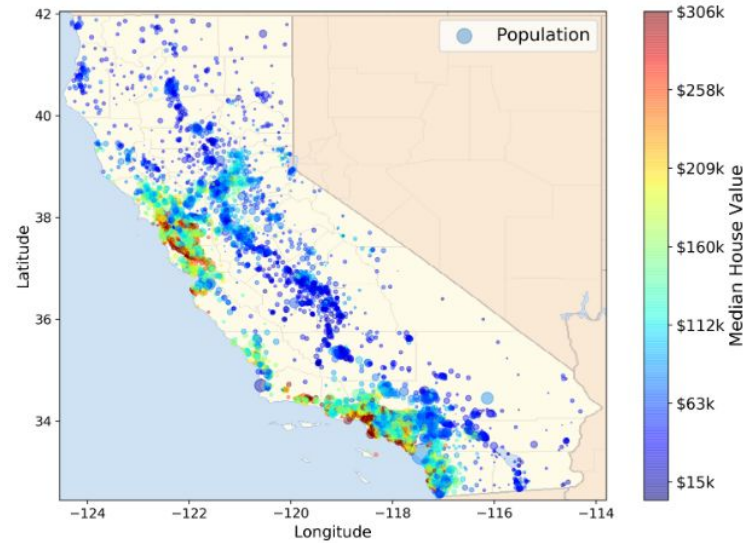


Figure 2-1. California housing prices

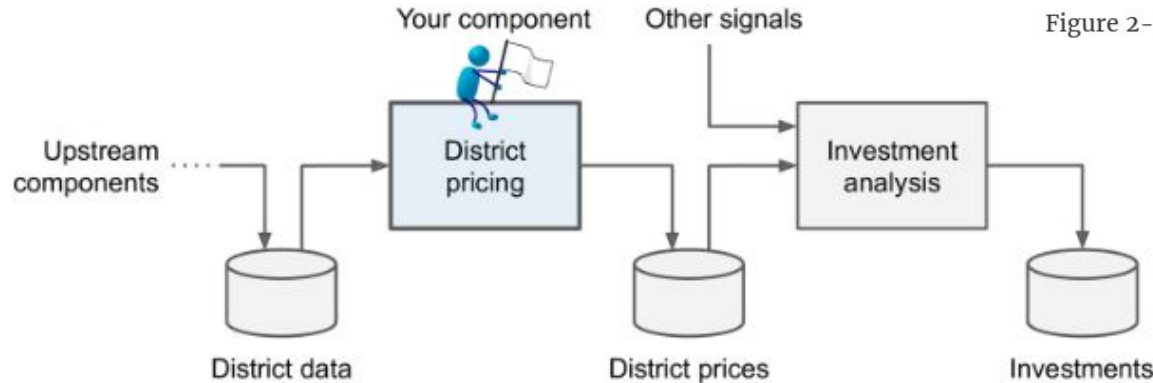


Figure 2-2. A Machine Learning pipeline for real estate investments

Performance measure : RMSE

Root Mean Square Error (RMSE)

Equation 2-1. Root Mean Square Error (RMSE)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

In [87]:

```
from sklearn.metrics import mean_squared_error

housing_predictions = lin_reg.predict(housing_prepared)
lin_mse = mean_squared_error(housing_labels, housing_predictions)
lin_rmse = np.sqrt(lin_mse)
lin_rmse
```

RMSE: Notations (1)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

- m is the number of instances in the dataset you are measuring the RMSE on.
 - For example, if you are evaluating the RMSE on a validation set of 2,000 districts, then $m = 2,000$.

RMSE: Notations (2)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

- $\mathbf{x}^{(i)}$ is a vector of all the feature values (excluding the label) of the i^{th} instance in the dataset, and $y^{(i)}$ is its label (the desired output value for that instance).

- For example, if the first district in the dataset is located at longitude -118.29° , latitude 33.91° , and it has 1,416 inhabitants with a median income of \$38,372, and the median house value is \$156,400 (ignoring the other features for now), then:

- $$\mathbf{x}^{(1)} = \begin{pmatrix} -118.29 \\ 33.91 \\ 1,416 \\ 38,372 \end{pmatrix}$$

and :

$$y^{(1)} = 156,400$$

RMSE: Notations (3)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

- $\mathbf{X}$ is a matrix containing all the feature values (excluding labels) of all instances in the dataset. There is one row per instance, and the i^{th} row is equal to the transpose of $x^{(i)}$, noted $(x^{(i)})^T$.
 - For example, if the first district is as just described, then the matrix $\mathbf{X}$ looks like this:

$$\circ \mathbf{X} = \begin{pmatrix} (x^{(1)})^T \\ (x^{(2)})^T \\ \vdots \\ (x^{(1999)})^T \\ (x^{(2000)})^T \end{pmatrix} = \begin{pmatrix} -118.29 & 33.91 & 1,416 & 38,372 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

RMSE: Notations (4)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

- $\mathbf{X}$ is a matrix containing all the feature values (excluding labels) of all instances in the dataset. There is one row per instance, and the i^{th} row is equal to the transpose of $x^{(i)}$, noted $(x^{(i)})^T$.
 - For example, if the first district is as just described, then the matrix $\mathbf{X}$ looks like this:

$$\circ \mathbf{X} = \begin{pmatrix} (x^{(1)})^T \\ (x^{(2)})^T \\ \vdots \\ (x^{(1999)})^T \\ (x^{(2000)})^T \end{pmatrix} = \begin{pmatrix} -118.29 & 33.91 & 1,416 & 38,372 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

RMSE: Notations (5)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

- h is your system's prediction function, also called a hypothesis. When your system is given an instance's feature vector $\mathbf{x}^{(i)}$, it outputs a predicted value $\hat{y}^{(i)} = h(\mathbf{x}^{(i)})$ for that instance ($\hat{y}$ is pronounced “y-hat”).
 - For example, if your system predicts that the median housing price in the first district is \$158,400, then $\hat{y}^{(1)} = h(\mathbf{x}^{(1)}) = 158,400$. The prediction error for this district is $\hat{y}^{(1)} - y^{(1)} = 2,000$.

Machine learning

source: Alexander Jung, “Machine Learning: The Basics”, 2022
([Available online](#))

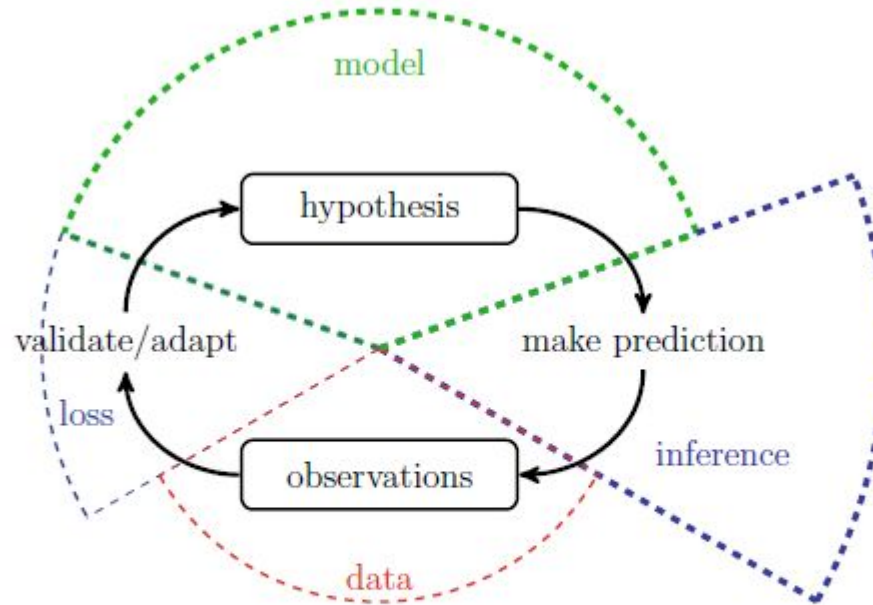


Fig. 1 Machine learning combines three main components: data, model and loss. Machine learning methods implement the scientific principle of “trial and error”. These methods continuously validate and refine a model based on the loss incurred by its predictions about a phenomenon that generates data.

RMSE: Notations (6)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

$\text{RMSE}(\mathbf{X}, h)$ is the cost function measured on the set of examples using your hypothesis h .

- Even though the RMSE is generally the preferred performance measure for regression tasks, in some contexts you may prefer to use another function. For example, suppose that there are many outlier districts. In that case, you may consider using the mean absolute error (MAE, also called the average absolute deviation).

Mean absolute error (MAE)

In [88]:

```
from sklearn.metrics import mean_absolute_error

lin_mae = mean_absolute_error(housing_labels, housing_predictions)
lin_mae
```

Equation 2-2. Mean absolute error (MAE)

$$\text{MAE}(\mathbf{X}, h) = \frac{1}{m} \sum_{i=1}^m |h(\mathbf{x}^{(i)}) - y^{(i)}|$$

Both the RMSE and the MAE are ways to measure the distance between two vectors: the vector of predictions and the vector of target values.

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

Setup (Chapter 2)

Importing common module.
Defining some attributes, functions.

```
# Python ≥3.5 is required
import sys
assert sys.version_info >= (3, 5)

# Scikit-Learn ≥0.20 is required
import sklearn
assert sklearn.__version__ >= "0.20"

# Common imports
import numpy as np
import os

# To plot pretty figures
%matplotlib inline
import matplotlib as mpl
import matplotlib.pyplot as plt
mpl.rcParams['axes', labelsizes=14]
mpl.rcParams['xtick', labelsizes=12]
mpl.rcParams['ytick', labelsizes=12]

# Where to save the figures
PROJECT_ROOT_DIR = "."
CHAPTER_ID = "end_to_end_project"
IMAGES_PATH = os.path.join(PROJECT_ROOT_DIR, "images", CHAPTER_ID)
os.makedirs(IMAGES_PATH, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = os.path.join(IMAGES_PATH, fig_id + "." + fig_extension)
    print("Saving figure", fig_id)
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

Downloading data (Chapter 2)

In [2]:

```
import os
import tarfile
import urllib.request

DOWNLOAD_ROOT = "https://raw.githubusercontent.com/ageron/handson-ml2/master/"
HOUSING_PATH = os.path.join("datasets", "housing")
HOUSING_URL = DOWNLOAD_ROOT + "datasets/housing/housing.tgz"

def fetch_housing_data(housing_url=HOUSING_URL, housing_path=HOUSING_PATH):
    if not os.path.isdir(housing_path):
        os.makedirs(housing_path)
    tgz_path = os.path.join(housing_path, "housing.tgz")
    urllib.request.urlretrieve(housing_url, tgz_path)
    housing_tgz = tarfile.open(tgz_path)
    housing_tgz.extractall(path=housing_path)
    housing_tgz.close()
```

In [3]:

```
fetch_housing_data()
```

In [4]:

```
import pandas as pd

def load_housing_data(housing_path=HOUSING_PATH):
    csv_path = os.path.join(housing_path, "housing.csv")
    return pd.read_csv(csv_path)
```

Looking at data structure (Chapter 2)

In [5]:

```
housing = load_housing_data()  
housing.head()
```

In [6]:

```
housing.info()
```

In [7]:

```
housing["ocean_proximity"].value_counts()
```

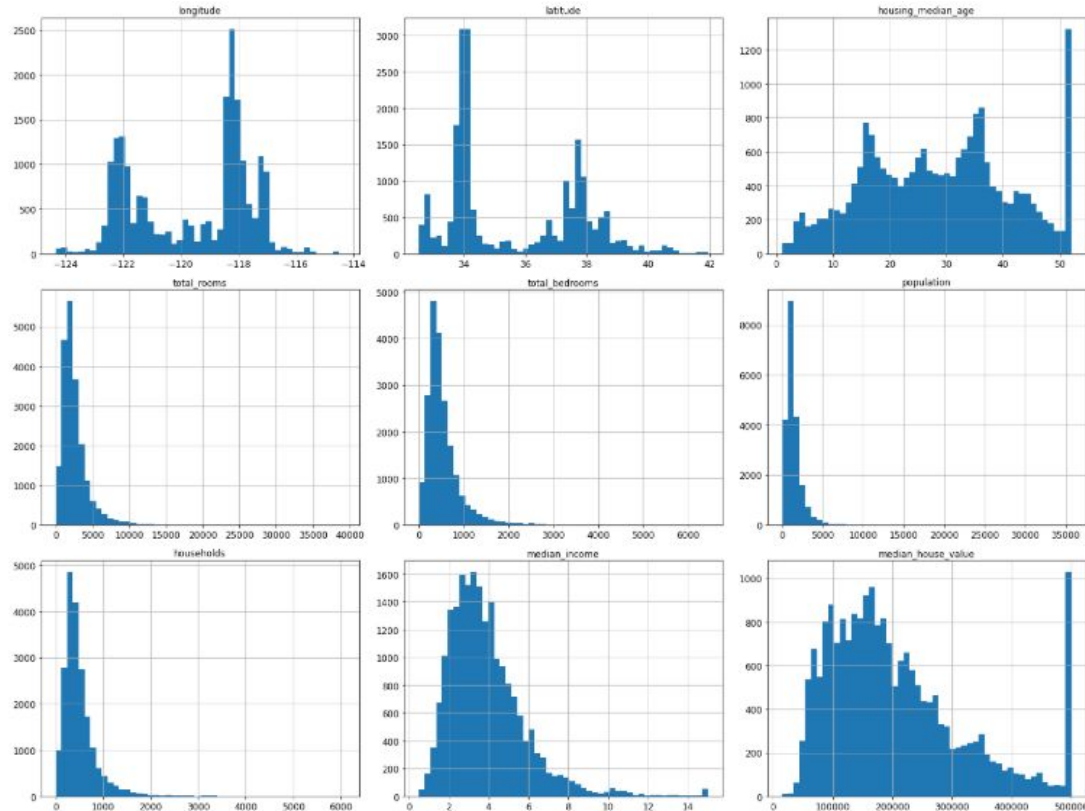
In [8]:

```
housing.describe()
```

In [9]:

```
%matplotlib inline  
import matplotlib.pyplot as plt  
housing.hist(bins=50, figsize=(20,15))  
save_fig("attribute_histogram_plots")  
plt.show()
```

Looking at data structure (Histograms) (1)



Looking at data structure (Histograms) (2)

1. First, the median income attribute does not look like it is expressed in US dollars (USD). After checking with the team that collected the data, you are told that the data has been scaled and capped at 15 (actually 15.0001) for higher median incomes, and at 0.5 (actually 0.4999) for lower median incomes. The numbers represent roughly tens of thousands of dollars (e.g., 3 actually means about \$30,000). Working with preprocessed attributes is common in Machine Learning, and it is not necessarily a problem, but you should try to understand how the data was computed.

Looking at data structure (Histograms) (3)

The housing median age and the median house value were also capped. The latter may be a serious problem since it is your target attribute (your labels). Your Machine Learning algorithms may learn that prices never go beyond that limit. You need to check with your client team (the team that will use your system's output) to see if this is a problem or not. If they tell you that they need precise predictions even beyond \$500,000, then you have mainly two options:

- A. Collect proper labels for the districts whose labels were capped.
- B. Remove those districts from the training set (and also from the test set, since your system should not be evaluated poorly if it predicts values beyond \$500,000).

Looking at data structure (Histograms) (4)

These attributes have very different scales, which may require feature scaling.

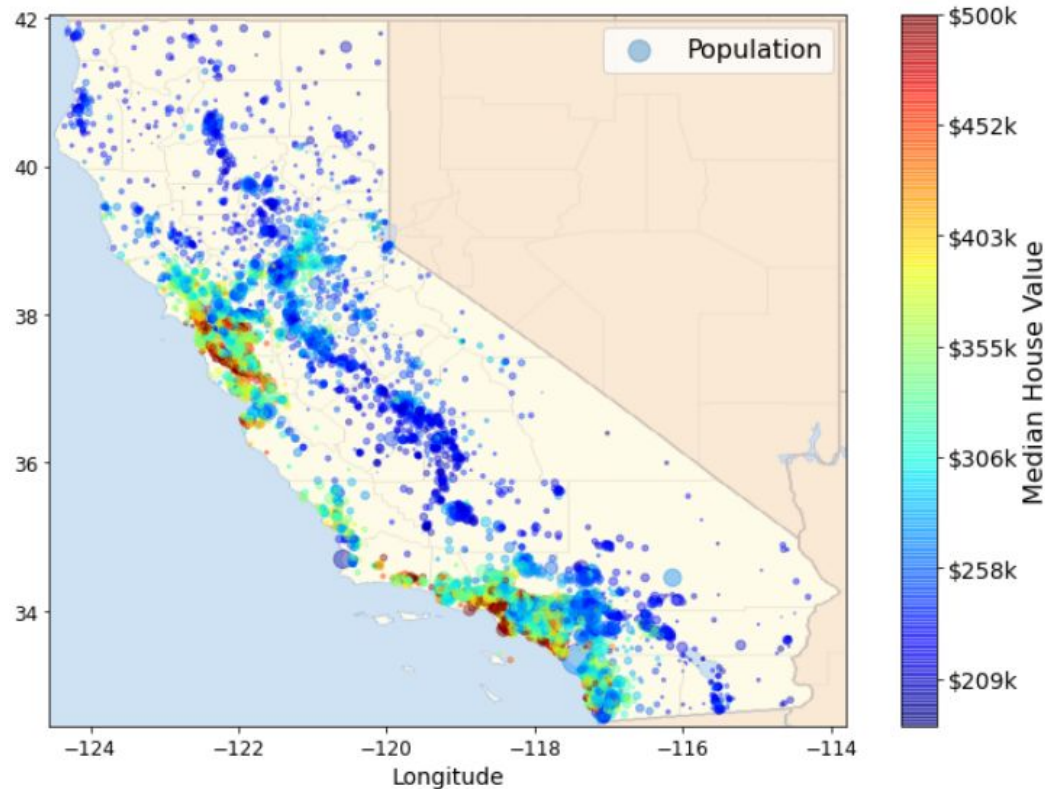
Finally, many histograms are *tail heavy*: they extend much farther to the right of the median than to the left. This may make it a bit harder for some Machine Learning algorithms to detect patterns. These attributes may be transformed to have more bell-shaped distributions.

Creating a test set (Chapter 2)

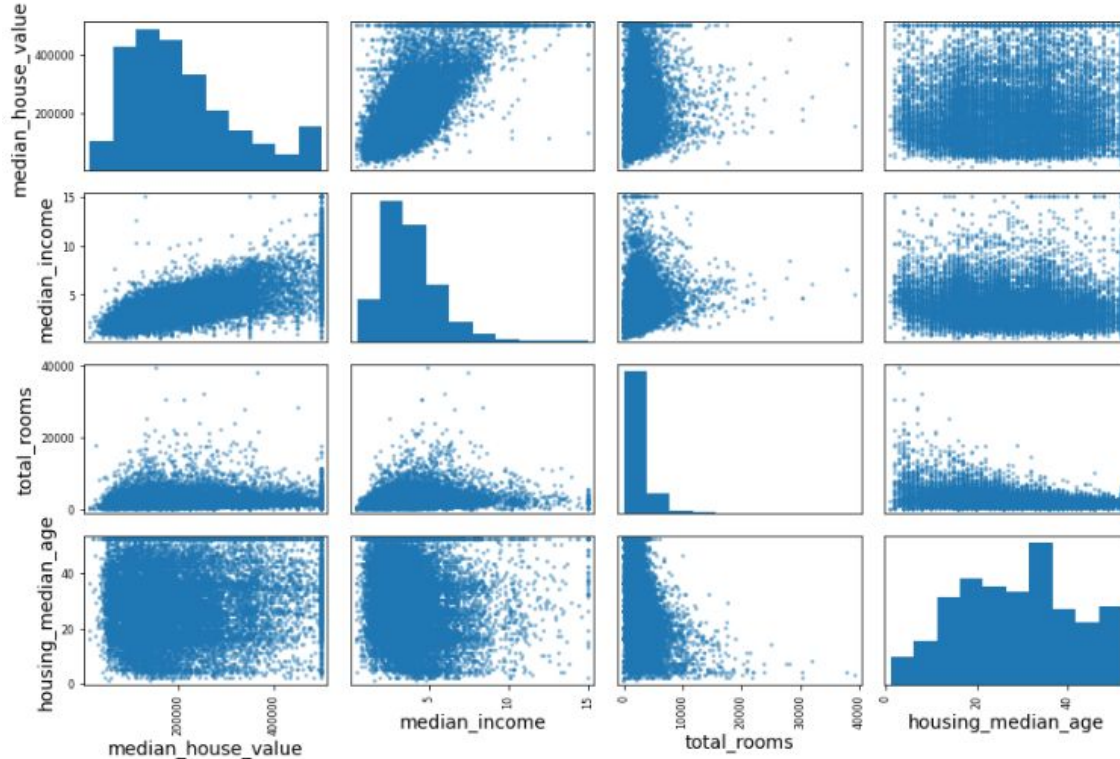
In [10]:

```
# to make this notebook's output identical at every run  
np.random.seed(42)
```

Visualizing to get insights (Chapter 2)



Looking for correlations (Chapter 2)



Then (Chapter 2)

- Preparing data
- Selecting and training a model
- Fine-tuning the model
- Evaluating the system

Trying other examples

Checking datasets

<https://www.kaggle.com/datasets?search=vibration&tags=12300-Engineering>

- Exploring
- Downloading data
- Looking at data structure

Checking dataset (1)

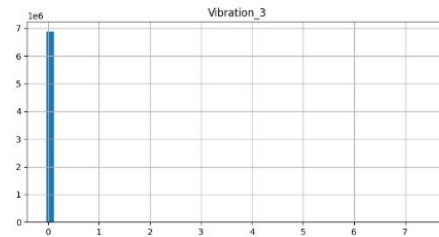
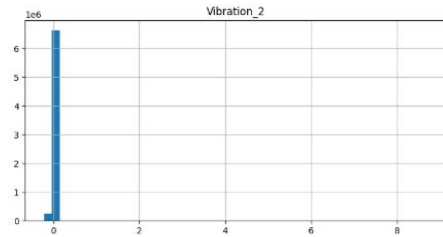
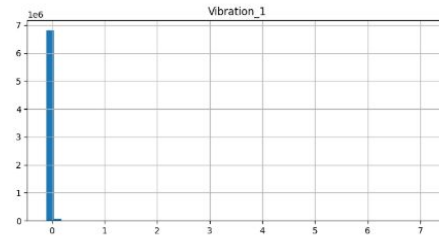
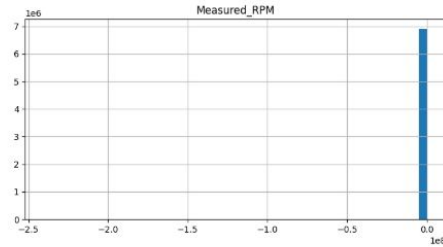
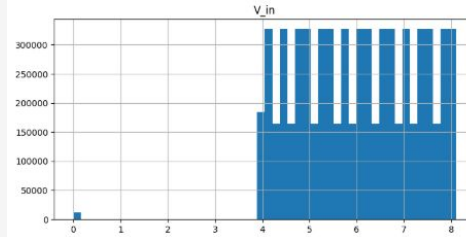
```
import pandas as pd
df= pd.read_csv('C:\\Users\\andreilo\\Desktop\\ToRead\\datasets\\4E.csv',encoding='latin-1')
df.head()
```

	V_in	Measured_RPM	Vibration_1	Vibration_2	Vibration_3
0	0.0	28.610235	0.0	0.0	0.0
1	0.0	28.610235	0.0	0.0	0.0
2	0.0	28.610235	0.0	0.0	0.0
3	0.0	28.610235	0.0	0.0	0.0
4	0.0	28.610235	0.0	0.0	0.0

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6914047 entries, 0 to 6914046
Data columns (total 5 columns):
#   Column      Dtype
---  -
0   V_in        float64
1   Measured_RPM float64
2   Vibration_1 float64
3   Vibration_2 float64
4   Vibration_3 float64
dtypes: float64(5)
memory usage: 263.8 MB
```

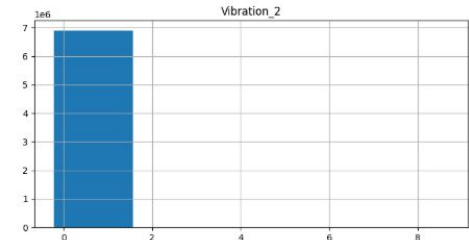
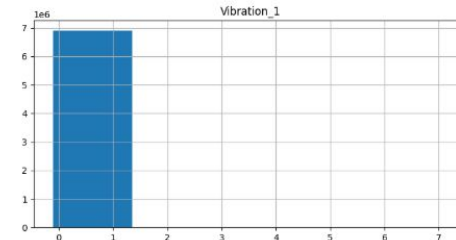
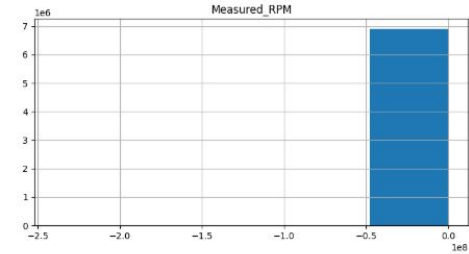
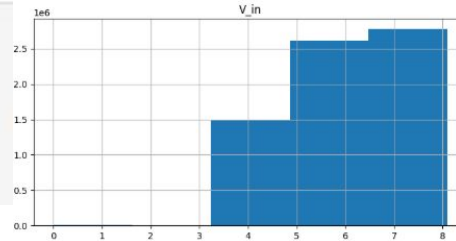

Checking dataset (2)

```
%matplotlib inline  
import matplotlib.pyplot as plt  
df.hist(bins=50, figsize=(20,15))  
plt.show
```



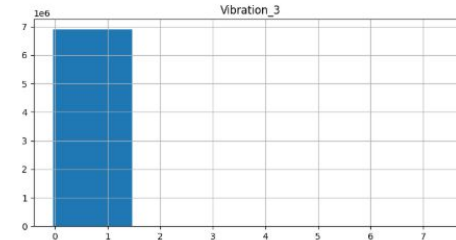
Checking dataset (3)

```
%matplotlib inline
import matplotlib.pyplot as plt
df.hist(bins=5, figsize=(20,15))
plt.show
```



```
df.describe()
```

	V_in	Measured_RPM	Vibration_1	Vibration_2	Vibration_3
count	6.914047e+06	6.914047e+06	6.914047e+06	6.914047e+06	6.914047e+06
mean	6.033176e+00	-1.406921e+05	3.856339e-03	7.944150e-03	7.550331e-03
std	1.240734e+00	5.839815e+06	9.451961e-02	1.639899e-01	1.151765e-01
min	0.000000e+00	-2.400000e+08	-1.012993e-01	-2.115834e-01	-3.185988e-02
25%	5.000000e+00	1.258990e+03	-6.020069e-03	-6.452799e-03	-1.237392e-03
50%	6.000000e+00	1.485709e+03	5.960464e-04	5.602837e-04	2.691746e-03
75%	7.100000e+00	1.713148e+03	7.945299e-03	8.031130e-03	7.190704e-03
max	8.100000e+00	1.963769e+03	7.162387e+00	8.686920e+00	7.464830e+00



Takeaways

- ML projects involve several decision-making points for analysing the data, preparing it, selecting the model and fine-tuning a solution.
- Interactive python notebooks in Jupyter (which can be also run on [NTNU server](#)) can support learning and teamwork in ML assignments.

Homework (by 23.01)

- Setup ML work environment
 - Communicate about any difficulties to TAs and/or Andrei.
- Try examples
 - Read chapters 1 and 2 in [the book](#).
 - From the book - do the [end-to-end ML project](#).
 - Look at existing datasets - try to extrapolate experience to one of those.